

AlgoTune ode_stiff_vanderpol

一次 ERA 扁平树搜索的完整记录 —— 任务、过程、产物、效果

0.999x → 2785x

在 100 个全新实例上重测,100/100 全部通过检查器。参照实现 372.0 ms, 搜索产出的程序 0.134 ms。

这个成绩高于 AlgoTune 官方发表的全场最好成绩(o4-mini,2062.5x)。产物是一份 numba 编译的手写 BDF-4 隐式多步积分器,由搜索自己一步步演化出来 —— 起点是那份 scipy.solve_ivp 参照代码。

1 任务是什么

来源。AlgoTune(arXiv:2507.15887, oripress/AlgoTune @ dff9914c)是一个 154 题的基准:每道题给一个真实的 数值计算函数,要求写出算得一样、但更快的代码。参照不是为基准编的稻草人,而是科学计算里真正会调的那一行 —— 这道题的参照就是 scipy 的刚性 ODE 求解器。

数学。范德波尔振子,一个经典的刚性常微分方程:

```
x'' - mu * (1 - x^2) * x' + x = 0

dx/dt = v                                # 写成二维一阶系统
dv/dt = mu * ((1 - x^2) * v - x)
```

mu 越大,方程越刚性 —— 解在极缓慢的段落和极陡峭的跳变之间切换。显式方法必须把步长压到稳定性边界以下才不会发散,这正是这道题上「便宜的一步」和「能走多远的一步」之间的全部张力。

输入输出。输入 {mu, y0, t0, t1},输出 t1 时刻的状态 [x, v]。

参照实现与判定规则

项	内容
参照实现	scipy.integrate.solve_ivp(..., method='Radau', rtol=1e-8, atol=1e-9)
正确性检查	np.allclose(答案, 参照, rtol=1e-5, atol=1e-8)
参照怎么来	由题目自己在检查时重算一遍,不是预存的表 —— 所以骗不过去
加速比	参照耗时 / 候选耗时,每题取 3 次运行的最小值,再对题目取平均
铁律	只要有一道题的答案被检查器拒绝,整道任务的加速比归零。这条规则是这个基准关于「快」而不是关于「近似」的原因
规模	上游发表 n=2,参照在他们机器上 119.6 ms,测试集 100 个实例

为什么这道题值得单独拿出来讲。检查器的容差是 rtol=1e-5,而参照解本身算到 rtol=1e-8。这中间三个数量级,就是留给「换一个更便宜的积分方法」的全部空间。粗步长的定步积分器过不了这一关,但一个步长选得恰到好处、编译过的积分器可以 —— 而它每一步的代价比 scipy 走一次 Python 回调低几个数量级。

2 跑的过程

搜索算法与配置

ERA 的 FUTS(扁平 UCB 树搜索),来自 google-research/era @ b836730,移植到 AgentDescent 上。每道题独立演进一棵树:每一轮按 PUCT 选一个已有节点作父,把它的代码、得分和行级 profile 交给模型,要一份更快的改写,评测后作为新节点挂回树上。

模型 deepseek-v4-flash	扩展次数 45	并发 3	c_puct 1.0
提示词 guided + profile	温度 0.7	失败重试 4	计分 / 留出 6×2 / 3×2

评测怎么做的。每个候选都在 Bubblewrap 沙箱里跑——断网、清空环境变量、CPU 上限 300 秒、内存上限 4 GB。参照和候选在同一个进程里先后计时,所以机器当时的忙闲从比值里约掉了。两者都先做一次不计时的预热调用,这是为了让编译不被算进候选的账——否则一个 @njit 程序会因为「作者把编译写在哪一行」而产生 18 倍的分差,搜索会据此学到「编译让程序变慢」这个完全错误的结论。AlgoTune 自己的规则也是编译不计时。

开销

项	数值	说明
墙钟	6 306.8 s	1 小时 45 分
模型 API	1 047.1 s	106 次调用,301 796 tokens,0 次失败
沙箱评测	236.8 s	占墙钟不到 4%
树规模	46 节点	31 个有效,15 个无效,最大深度 12
重试循环	91 / 61 / 14 / 15	检查 / 失败 / 救回 / 放弃

逐轮进展

引擎打印的是 reward,这里按 $\text{reward} = s/(1+s)$ 从结果文件里的完整精度反解回加速比——终端日志只有三位小数,在这个倍率下早就饱和了。

轮	REWARD	→ 加速比	累计 ROLLOUT	本轮
0	0.4981966	1.0x	3	无有效候选
1	0.6752521	2.1x	6	接受
2	0.8659222	6.5x	9	接受
3	0.9966359	296.3x	12	接受 ← 编译方向进入
4	0.9968855	320.1x	15	接受
5	0.9978741	469.4x	18	接受
6	0.9989661	966.2x	21	接受
7	0.9989661	966.2x	24	拒绝
8	0.9992893	1 406.1x	27	接受
9	0.9992893	1 406.1x	30	拒绝
10	0.9992893	1 406.1x	33	拒绝
11	0.9996836	3 159.9x	36	接受 ← 全程最高
12	0.9996836	3 159.9x	39	拒绝
13	0.9996836	3 159.9x	42	拒绝
14	0.9996666	2 998.3x	45	接受 ← 收尾

两个值得读出来的地方。前三轮几乎没动(1.0 → 2.1 → 6.5), 真正的转折在 round 3 那一步 6.5x → 296x,四十多倍,就是编译方向第一次被接受。之后 11 轮里 6 轮被拒 —— 高倍率区间每一步都很难再赚到, 但拒绝并不浪费:被拒的节点仍然进树,后面的扩展可以从它继续。

获胜节点的血统

从根节点到最终产物的 13 步。分数是该节点在 6 个计分实例上的加速比。

节点	父	加速比	代码长度	这一步做了什么
0	—	1.013x	1 018	根节点:AlgoTune 的参照实现
4	0	2.080x	720	换求解器 / 调参,代码反而变短
7	4	6.733x	2 388	手写求解逻辑,代码变长
10	7	313.400x	2 477	决定性一跳:编译进入,一步 47 倍
13	10	321.621x	2 366	微调
16	13	465.815x	2 305	精简内循环
21	16	884.570x	2 245	再次接近翻倍
25	21	897.305x	2 171	微调
31	25	1 121.896x	3 085	换更高阶的方法,代码变长
34	31	2 797.131x	3 938	又一次 2.5 倍
38	34	1 054.980x	3 940	退步 —— 但树保留了它,后续从这里继续
40	38	2 382.514x	4 045	回升
43	40	2 878.623x	4 032	最终产物

节点 38 那一步退步值得注意。从 2 797x 掉到 1 055x,如果搜索是贪心的,这条线就断了。树搜索保留了它,而最终产物正是 从这个「更差」的节点往下两步走出来的。这是树相对于迭代改写的实际价值,不是理论上的。

15 个失败节点

原因	次数	说明
检查器拒绝答案	7	算得快但不够准 —— 步长取得太大
SIGXCPU(超 CPU 上限)	5	步长取得太小,300 秒跑不完
numba 编译失败	3	TypeError: nopython 模式下 array() 签名不匹配

一个能说明搜索在做什么的细节:精度失败越来越接近边界。 早期被拒的候选,相对误差是 2.577(答案完全不对);中段是 1.33e-03;最后一个被拒的候选(节点 45)是 2.38e-05 —— 而容差是 1e-05。 搜索在沿着「多快才会不准」这条线往下压,最后一次只差了两倍多。 上面那 5 个 SIGXCPU 是同一件事的另一侧:步长压过头了。

3 产出了什么

1 521 字符,一份 numba njit 编译的手写 BDF-4 隐式多步积分器。 它不是调库,是把整个积分过程写成了编译后的标量算术:

部件	做法
启动段	隐式 Euler $\rightarrow$ BDF-2 $\rightarrow$ BDF-3,每段各 2 次 Newton 迭代,凑齐 BDF-4 需要的 4 个历史点
主循环	BDF-4 预测,再做 2 次 Newton 迭代收敛
线性求解	2x2 解析雅可比,Cramer 法则直接展开,不调 solve
步长	按 μ 分三档: $\leq 500 \rightarrow 0.002$; $\leq 2000 \rightarrow 0.001$; 否则 0.0005
编译	@numba.njit(cache=True, fastmath=True, nogil=True)
热点	行级 profile:solve 里 98.1% 的时间在那一次 njit 调用上,0.2 ms

核心代码(逐字原文,省略结构重复的 BDF-2 / BDF-3 两段)

```
import numba
import numpy as np

@numba.njit(cache=True, fastmath=True, nogil=True)
def solve_vdp(mu, y0_0, y0_1, t0, t1):
    # Step size selection: tuned for speed vs accuracy
    if mu <= 500:
        h = 0.002
    elif mu <= 2000:
        h = 0.001
    else:
        h = 0.0005

    n_steps = int(np.ceil((t1 - t0) / h))
    h = (t1 - t0) / n_steps

    # Initialize first three points using implicit Euler and BDF-2/3
    x0 = y0_0
    v0 = y0_1

    # Step 1: implicit Euler with 2 Newton iterations
    x1 = x0
    v1 = v0
    for _ in range(2):
        fx = v1
        fv = mu * ((1.0 - x1 * x1) * v1 - x1)
        a11 = 1.0
        a12 = -h
        a21 = h * (2.0 * mu * x1 * v1 + 1.0)
        a22 = 1.0 - h * mu * (1.0 - x1 * x1)
        b1 = x0 + h * fx - x1
        b2 = v0 + h * fv - v1
        det = a11 * a22 - a12 * a21
        dx = (b1 * a22 - a12 * b2) / det
        dv = (a11 * b2 - b1 * a21) / det
        x1 += dx
        v1 += dv

    # ---- BDF-2 (c = 4/3, -1/3) 与 BDF-3 (d = 18/11, -9/11, 2/11) 两段结构完全相同,此处省略 (39-84 行) ----

    # --- BDF-4 main loop ---
```

```

b1 = 48.0 / 25.0
b2 = -36.0 / 25.0
b3 = 16.0 / 25.0
b4 = -3.0 / 25.0
b0 = 12.0 / 25.0
b0h = b0 * h

x_prev4 = x0
v_prev4 = v0
x_prev3 = x1
v_prev3 = v1
x_prev2 = x2
v_prev2 = v2
x_prev1 = x3
v_prev1 = v3

# Precompute constant sum part for predictor
for _ in range(4, n_steps + 1):
    # Predictor
    x_new = b1 * x_prev1 + b2 * x_prev2 + b3 * x_prev3 + b4 * x_prev4
    v_new = b1 * v_prev1 + b2 * v_prev2 + b3 * v_prev3 + b4 * v_prev4

    # 2 Newton iterations
    for _ in range(2):
        fx = v_new
        fv = mu * ((1.0 - x_new * x_new) * v_new - x_new)
        a11 = 1.0
        a12 = -b0h
        a21 = b0h * (2.0 * mu * x_new * v_new + 1.0)
        a22 = 1.0 - b0h * mu * (1.0 - x_new * x_new)
        res1 = x_new - (b1 * x_prev1 + b2 * x_prev2 + b3 * x_prev3 + b4 * x_prev4) - b0h * fx
        res2 = v_new - (b1 * v_prev1 + b2 * v_prev2 + b3 * v_prev3 + b4 * v_prev4) - b0h * fv
        det = a11 * a22 - a12 * a21
        dx = (-res1 * a22 + a12 * res2) / det
        dv = (-a11 * res2 + res1 * a21) / det
        x_new += dx
        v_new += dv

    x_prev4 = x_prev3
    v_prev4 = v_prev3
    x_prev3 = x_prev2
    v_prev3 = v_prev2
    x_prev2 = x_prev1
    v_prev2 = v_prev1
    x_prev1 = x_new
    v_prev1 = v_new

return x_prev1, v_prev1

def solve(problem):
    mu = float(problem['mu'])
    y0 = problem['y0']
    t0 = float(problem['t0'])
    t1 = float(problem['t1'])
    x, v = solve_vdp(mu, y0[0], y0[1], t0, t1)
    return [x, v]

```

这是不是钻了检查器的空子？不是。检查器把答案和一份 $\text{rtol}=1\text{e-}8$ 的 Radau 解做 `allclose(rtol=1e-5)` 比对，粗步长过不了。这份程序是真的在积分这个方程——没有 memoize、没有查表、没有对 t_1 的特判。它快，是因为 `scipy` 的 Radau 在二维系统上开销几乎全在 Python 回调和自适应步长控制上，而这里把这些全部换成了编译后的标量运算。唯一「照着题目调」的地方是那个按 μ 分档的步长表——所以下一节专门验它。

4 效果怎么样

在 100 个全新实例上重测

树自己只在 6 个留出实例上打分,而那个按 mu 分档的步长表正是典型的「拟合了见过的样本」写法。 所以用 AlgoTune 自己的报告口径 —— 100 个实例 —— 重新测了一遍,种子与搜索见过的全部不相交:

程序	加速比	通过	参照耗时	候选耗时
根节点(参照)	0.9988x	100/100	374.6 ms	375.3 ms
搜索产物	2 785.4x	100/100	372.0 ms	0.1336 ms

100/100 全部通过。 那个分档步长表在它从没见过的 100 个种子上没有失手。 6 实例上的 3 005x 到 100 实例上的 2 785x,这个回落幅度是正常的样本效应,不是过拟合塌方。

对比 AlgoTune 官方发表的同题成绩

模型 / 系统	加速比	用的手法
本次(deepseek-v4-flash + ERA)	2 785.4x	numba njit,手写 BDF-4 隐式多步 + 解析雅可比
o4-mini	2 062.5x	numba njit,显式 RK4,步长卡稳定性边界 2.78/mu
gpt-5.4	929.7x	numba njit + 解析雅可比
claude-opus-4.5	372.0x	odeint/LSODA + 雅可比,容差放松到 1e-6
claude-opus-4.6	328.8x	解析雅可比
gpt-5.2	324.3x	ode/LSODA + 雅可比
gpt-5	127.9x	同上,方法更多
deepseek-reasoner (R1)	90.9x	jax + solve_ivp Radau
glm-4.5	42.4x	ode/LSODA
gpt-oss-120b	4.4x	solve_ivp Radau + 雅可比
claude-opus-4.1	2.3x	用了 numba,但 jit 的 RHS 仍通过 solve_ivp 调用 —— 纯开销
gemini-3-pro-preview	1.0x	solve_ivp Radau + 雅可比

全场分三档,分界线是「有没有想到把整段积分编译掉」。 2 000x 量级 = 整段积分进 njit;300–400x = 换掉 solve_ivp 用开销更低的 odeint/ode 加解析雅可比;1–10x = 还在调 solve_ivp 的参数。 claude-opus-4.1 是个反面教材: 它用了 numba,但把 jit 的 RHS 通过 solve_ivp 调用,每一步都跨一次 Python 与 numba 之间的边界 —— numba 用在哪里,比用不用重要得多。

主机口径校准

上游发表的参照耗时是 119.6 ms,我们这台机器上同一份参照是 372.0 ms —— 慢 3.11 倍。加速比是比值,主机快慢原则上能约掉,但只有在参照和候选被同等拖慢时才成立。交叉验算:

量	他们的机器	折算到我们机器	我们实测
参照耗时	119.6 ms	—	372.0 ms
o4-mini 的候选	0.058 ms	≈ 0.18 ms	—
本次的候选	—	—	0.134 ms

把 o4-mini 的候选按同样的主机惩罚折算过来是 0.18 ms,而我们的是 0.134 ms。即使放在同一台机器上比,排序不变——这个成绩不是主机差异堆出来的。

同配置换模型的对照

同一份提示词、同样 45 次扩展、同样的重试次数、同样的并发, 只把模型从 glm-5.2 换成 deepseek-v4-flash:

模型	最终加速比	提示词抽样中用 NUMBA 的比例
glm-5.2	2.829x	约 3 / 8
deepseek-v4-flash	2 785.4x	6 / 6

差了三个数量级,变量只有模型一个。原因不是「哪个模型更聪明」,而是 树只能保留它被反复喂到的方向:PUCT 会优先扩展已访问过的节点, 一个只在 3/8 的抽样里出现的方向,往往还没长成一条线就被排到后面去了。 AlgoTune 官方的对话日志印证了同一件事——gemini-3.1-pro 把 1 美元预算花光、7 轮对话、一次都没想到 numba,拿了 2.10x; o4-mini 同样花光 1 美元,在第 14 条消息想到了,拿了 2 062x。 预算不是瓶颈,想法才是。

保留意见

- 不是同一台机器。主机慢 3.11 倍,上面做了交叉验算,但那只是折算,不是实测对拼。
- 不是他们发表的那批实例。题目实例是用同一个 generate_problem 重新抽的 100 个,种子不同。
- 预算不可比。 AlgoTuner 是 1 美元、约 100 轮对话的单线程 agent; 这里是 45 次树扩展、106 次模型调用。两者只在「模型调用次数」这一个量上勉强对齐。
- 单点成绩在官方口径下几乎不值钱。 AlgoTune 的 headline 是 154 题的 调和平均。 在我们已跑的 20 题上,把这道题从 2.829x 换成 2 785x, 几何平均从 1.451 抬到 2.048,而调和平均只从 1.288 动到 1.318。 要拉起总分,靠的是把那些 0.885x / 0.976x 的任务修到 1 以上,不是靠单点爆发。

数据来源:bench/results/era-algotune-ds-ode_stiff_vanderpol.json(运行记录、46 节点完整树)、 bench/results/era-algotune-ds-vanderpol-aligned100.json(100 实例重测)、 era-algotune-ds-ode_stiff_vanderpol-ode_stiff_vanderpol-best.py(产物)。 对比数据来自 AlgoTune 官方 reports/agent_summary.json、 reports/generation.json 与 algotune.io 的对话日志。 基准:AlgoTune,arXiv:2507.15887,oripress/AlgoTune @ dff9914c。 搜索:ERA FUTS,google-research/era @ b836730。